My Current Mental Model *of GPU Hierarchy*

The logic in this document came from 7 weeks of kernels breaking the framework. Each one broke something I thought was general. The formatting is from an LLM. This is v8 of a mental model that is still changing.

📓 NOTE ON THIS DOCUMENT

The framework logic here came from tracing index bugs in tiled GEMM, dense attention, and paged KV MLA. Each new kernel broke something I thought was general. I found the minimal fix each time, and the framework is what accumulated across those fixes. The CSS, tables, and prose structure were generated by an LLM from my bullet-point notes. I've done a pass to align the voice. This is a bookmark — a clean snapshot of v8. It will change.

THE SEQUENCE AT A GLANCE

1a · Dimension Fate → ↳ Algorithm Classification → 1b · Access Patterns →
2 · Binding Table → 4a · Lifetime (block) → 3 · FSM → opt · Pipeline →
4b · Lifetime (thread) → 5 · Index Trees → Code Derivation

When we write a GPU kernel, two separate sets of decisions are happening at the same time. One is about the algorithm — what mathematical operation is being performed, and how does it work? The other is about the architecture — where does each piece of data live during execution, which threads handle which work, and when does data move between memory levels?

When we often start learning parallel computing, we often focuse entirely on the first set and guessed at the second. That produces kernels that give correct answers but run far slower than the hardware is capable of. The data lives in the wrong places, moves at the wrong times, and gets shared through the wrong channels.

This framework is a structured way to make those architectural decisions on paper before writing any code. It covers kernels where every contracted dimension uses an associative and commutative operator — which turns out to be a wide and important class. Each question builds directly on the answers before it. By the time we open our editor, every structural decision is already made. We're just translating a completed design into code.

The framework has five numbered steps, but Step 1 has a decision point in the middle. We work through Step 1a, make a classification decision, then complete Step 1b. Understanding why the steps are in this order is what makes the framework work on kernels we've never seen before. That's what I've tried to explain here.

FRAMEWORK SCOPE — THE ALGEBRAIC PROPERTY THAT MAKES DERIVATION MECHANICAL

I've used this framework to derive kernel architectures for operations where every REDUCE dimension uses an associative and commutative operator — specifically vector add, transpose, shared reduced sum, GEMM, dense attention, paged KV MLA, and sparse attention. The operators that fit this pattern include addition, multiplication, max, min, bitwise AND/OR/XOR, dot product, and online softmax merge — any body where partial results can be combined in any order and produce the same final answer.

That algebraic property is what makes the derivation mechanical. Because partial results can be combined in any order, the framework can assign REDUCE dimensions to any level of the hierarchy — warp shuffle, block merge, or sequential thread-private accumulator — and the partial results compose consistently regardless of which threads produced them. The binding table, FSM, lifetime tables, and index trees all follow from this single property. Without it, the derivation method breaks down.

If a REDUCE dimension uses an operator that isn't associative and commutative — subtraction, selection (top-K), scan (prefix sum) — the framework is probably at its edge for that operation. Those tend to require different phase structures and intermediate tensors that the binding table isn't set up to derive. Catching that at Step 1a saves a lot of wasted work further in.

STEP 1A

Dimension Fate

"What does the math do to each dimension — does it survive into the output, or disappear?"

This is always the first thing we do with any tensor operation. We don't think about threads yet, we don't think about memory yet — we look only at the mathematical operation and ask a single precise question about each dimension: does this dimension appear in the output tensor, or does it get consumed by the computation?

To answer this, write down the shape of every input tensor and the shape of every output tensor. Then go through each dimension by name and find it in the output. It's either there, or it isn't.

> What counts as an output tensor? Only tensors that are written to global memory at the end of the kernel. If the operation produces an intermediate value that is consumed within the same kernel and never written out — a running max for numerical stability, a partial dot product, a row sum used to normalize — it is an *internal intermediate*, not an output tensor. Do not list it here. Internal intermediates appear later as state variables in the REDUCE dimension's description, or as temporary smem buffers in Step 2. If we're unsure whether something is an output or an intermediate, ask: "does the caller of this kernel read this value?" If yes, it is an output. If no, it is an intermediate.

SURVIVED DIMENSIONS — AREA

A dimension that appears in the output is called an AREA dimension. It means the operation produces one independent result for every position along this dimension. Computing the result at position 3 requires no knowledge of what happened at position 7. These positions are completely independent of each other.

This independence is valuable because it means we can assign different blocks of threads to different positions along an AREA dimension, and those blocks never need to communicate. AREA dimensions are what make parallelism cheap.

CONTRACTED DIMENSIONS — REDUCE

A dimension that does *not* appear in the output is called a REDUCE dimension. It means all the values along that dimension are being combined into a smaller result — summed, compared, selected from, or otherwise collapsed. The dimension gets consumed by the computation.

We can still parallelize along a REDUCE dimension, but the parallelism costs something: the threads that processed different pieces of it must eventually share their partial results and combine them. This combination step is called a *reduction*, and it requires synchronization. REDUCE dimensions are what make GPU design interesting.

Once we have identified every REDUCE dimension, write down the reduction body for each one in plain language — the specific operation that combines elements along that dimension. Is it addition? A maximum? A comparison-and-swap? A heap insertion? Be specific, because the body operation determines what state variable a thread needs to carry through the computation.

That state variable is the piece of information a thread holds from one element to the next as it processes a REDUCE dimension. For a running sum, the state variable is a single number initialized to zero. For a running maximum, it's a single number initialized to negative infinity. For a heap-based selection, it's the heap itself — an array of k slots, each holding a (value, position) pair. Write down the type of the state variable and how it's initialized, because this will appear directly in later steps.

> A common mistake: skipping the state variable description because "it's obvious." It rarely is, and the state variable type is what determines how much register space a thread needs and whether the reduction can be done with warp shuffles or requires shared memory. Write it down explicitly.

REDUCTION CHARACTER — THE SINGLE GATE BEFORE CONTINUING

Alongside the reduction body and state variable, verify the reduction character of each REDUCE dimension. This framework covers exactly one character:

Parallel — the only character this framework handles. The body operation is associative and commutative. This means any two partial results can be combined in any order and produce the same final answer. This algebraic property is what makes the binding table, FSM, and warp shuffle derivation mechanical: because order of combination doesn't matter, we can freely assign slices of the REDUCE dimension to different threads, warps, and blocks without coordinating which slice each one processes. Warp shuffle reductions, block-level binary trees through shared memory, and sequential thread-private accumulators are all correct implementations of the same associative+commutative reduction — the framework tells us which level to use and how to connect them. Common bodies with this character: addition, max, min, product, bitwise AND/OR/XOR, dot product, online softmax merge (the fmaxf + rescaled sum combination), and any weighted sum.

If a REDUCE dimension is not associative and commutative — for example, selection (top-K, argmax) or scan (prefix sum, running count) — the framework is probably at its edge for that operation. Those use different parallel algorithms (warp min-heaps, radix select, Blelloch scan) that introduce intermediate tensors and phase structures I haven't been able to fit into this derivation. They're worth a different starting point.

> Quick character lookup — parallel bodies covered by this framework:
>
> Parallel (proceed): sum · product · max · min · bitwise AND/OR/XOR · count of matching elements · weighted sum · dot product · online softmax merge (max + rescaled sum) · any body where partial results can be combined in any order regardless of which threads produced them.
>
> Not sure? Ask: "if I give two threads half the data each, can they each produce a partial result that I can combine afterward without needing to see the full dataset?" If yes, parallel — proceed. If the merge step requires knowing the full dataset (e.g. the k-th boundary score in selection, or all prior outputs in a scan), that operation is likely outside what this derivation can handle — worth noting before going further.

```
Feeds into final step → Symbol legend (state variable names and init values, one row per REDUCE dim) · Phase skeleton (COMPUTE phase content: the accumulation body) · Declaration table (register state variable declarations)
```

PIVOT INSIDE STEP 1

Algorithm Classification

"Which algorithm family implements this reduction, and what does the output need to satisfy?"

WHY THIS FIRES HERE — NOT AT THE BEGINNING

This decision needs two things as input: the body operation of the REDUCE dimension (what are we combining?), and the relative sizes of the dimensions involved (how much data is being reduced, and how large is the result?). Neither of those is available before Step 1a. We can't classify the algorithm before we know what the algorithm is doing.

This decision also needs to happen *before* Step 1b, because the algorithm we choose determines which tensors exist in your kernel. A multi-pass algorithm has intermediate tensors that a single-pass algorithm doesn't. Analyzing access patterns before knowing which algorithm we're using means you might analyze the wrong tensors entirely.

Look at the REDUCE dimension and its body operation — which we have already verified is associative and commutative. Ask yourself: is there a known algorithm family that implements this body efficiently on a GPU? Most parallel reductions fall into one of a small number of families. Simple reductions — sum, min, max — use a parallel reduction tree. Matrix multiplications use

tiled GEMM. Attention uses fused online softmax. Each family has its own well-studied structure that determines what intermediate tensors exist and how the FSM phases are ordered.

For each candidate family, consider the relevant problem dimensions. If there are multiple family options, the right one is usually determined by a ratio or threshold — the size of the output relative to the size of the input, the sparsity of the data, or a threshold below which one approach dominates and above which another does. Write down the ratio, compute it with your actual dimension sizes, and commit to a specific family. Then name it explicitly at the top of your derivation.

Not recognizing an algorithm family for the operation is a useful signal — it's happened to me several times during this competition, and it usually means the algorithmic landscape for that problem is worth understanding before the derivation starts. The framework takes over once a family is chosen, but the choice itself has to come from somewhere outside it.

> Common algorithm families — parallel reductions only. All entries below have associative and commutative reduction bodies. Match your reduction body to the closest entry, then use the listed family name in your derivation record. Operations outside this list (prefix scan, selection/top-K, sort) are outside this framework's scope.
>
> Parallel reduction (sum / max / min / AND / OR): Family = parallel reduction tree. Warp-level: shuffle-down. Block-level: binary tree through smem. No intermediate tensors.
>
> Matrix multiply (C = A × B, reduction over inner dimension K): Family = tiled GEMM. Tiles of A and B staged in smem; each thread accumulates a private output tile. No intermediate tensors unless split-K is used.
>
> Attention (softmax-weighted sum of V, reduction over sequence length J and head dim K): Family = Flash Attention (fused online softmax). Intermediate tensors: running state (m, l, o) in registers per query; per-tile scores in smem. No S[N,N] matrix. The online softmax merge — fmaxf + rescaled sum — is parallel character.
>
> Softmax (normalize a vector, reduction over the vector dimension): Family = two-pass (max then exp+sum) or online single-pass. Intermediate tensors: row max and row sum if two-pass; online state (m, l) if single-pass.
>
> Layer norm / RMS norm (normalize across hidden dim): Family = two-pass reduction (mean/variance then normalize). Intermediate tensors: mean, variance per row.

> Histogram (count occurrences, reduction over input positions): Family = privatized histogram (one histogram copy per warp or block, then merge). Warp primitive: match (__match_any_sync) to avoid serialization at bin collisions.
>
> Convolution (sum over kernel spatial positions and input channels): Family = direct tiled convolution (like GEMM but with filter reuse) or implicit GEMM (reshape to matrix multiply). Choose based on whether input padding is needed and filter size.

Output contract. Before moving on, also write down what properties the output must satisfy beyond just having the right shape. These constraints are part of the problem specification but are easily forgotten until they surface as bugs late in development. Does the output need to be sorted in a particular order? Do values need to sum to one, indicating renormalization is required? Is the output a single randomly sampled index rather than a set of values? Does the operation need to be numerically stable or deterministic across runs? Whatever the answer, write it down — then immediately name the FSM phase each constraint generates. A renormalization requirement becomes a NORMALIZE phase. A sort requirement becomes a SORT phase before STORE. A numerical-stability contract becomes a running-max initialization in INIT. Recording the contract and the phase it produces together prevents the class of bug where a contract requirement is written down in Algorithm Classification and then forgotten before Step 3.

WHAT THIS STEP PRODUCES — FILL IN ONE ROW PER REDUCE DIMENSION

FIELD	EXAMPLE — TILED GEMM	EXAMPLE — FLASHINFER SPARSE ATTENTION	YOUR KERNEL — WHAT TO WRITE HERE
Algorithm family	*tiled GEMM (outer-product accumulation)*	*block-sparse Flash Attention (fused online softmax over nonzero KV blocks only)*	*Which named family from the reference list handles your REDUCE body? Commit to one name explicitly — "tiled GEMM", "parallel reduction tree", "Flash Attention", etc. All families must have parallel-character reductions.*

FIELD	EXAMPLE — TILED GEMM	EXAMPLE — FLASHINFER SPARSE ATTENTION	YOUR KERNEL — WHAT TO WRITE HERE
Intermediate tensors	*none — accumulation is register-private*	*kv_indices[nnz_blocks] — sparse KV block index array (lookup leaf for j); row_ptr[num_q_blocks+1] — CSR row pointers; score_smem[T_J] per tile; (m, l, o) softmax state in registers*	*List every tensor your chosen family introduces that was not in the input/output list. These must appear in Step 1b's access pattern table. Write "none" if the family adds no new tensors.*
Output contract	*$C[m,n] = \Sigma_k A[m,k]\cdot B[k,n]$; no normalization; FP associativity acceptable*	*O[q,:] = normalized sparse-softmax-weighted sum over attended KV positions only; weights sum to 1 over attended set; numerically stable via running max subtraction*	*What properties must the output satisfy beyond shape? Be specific. Vague "correct values" is not useful. Include normalization, ordering, stability, and determinism requirements.*
FSM phases from contract	*STORE only (no normalization or ordering required)*	*NORMALIZE (÷l_acc for softmax), INIT with m=−∞ (numerical stability), SPARSE_ITER phase (walk kv_indices[row_ptr[q]:row_ptr[q+1]] instead of sequential j tiles)*	*For each contract requirement, name the FSM phase it generates. Every phase named here must appear in Step 3's FSM table — this list seeds the phase sequence.*

Feeds into final step → Phase skeleton (tile loop structure from family; novel phase rows from FSM phases column) · Declaration table (intermediate tensor declarations) · Index expression table (intermediate tensor accesses must be analyzed)

STEP 1B

Access Patterns

"How is each tensor actually read or written — what kind of indexing does each dimension use?"

With the algorithm chosen, we can complete the geometry step by examining how each tensor is accessed. For every dimension of every tensor — inputs, outputs, and any intermediate tensors your chosen algorithm introduces — we assign a leaf type that describes how the index into that dimension is computed. This classification matters because different leaf types have different performance characteristics and generate different code.

> Two categories of leaves. Every index expression contains two kinds of leaves: *execution leaves* that describe how an address is computed from thread and loop variables, and *value leaves* that describe what happens to the data once we reach it. Classify both for every access.

— Execution leaves —	*Describe how the physical address is computed from execution context.*
Arithmetic	The address is a linear combination of execution variables: `base + i₁·s₁ + i₂·s₂ + …` using only addition and multiplication. The default and best case — the hardware can predict these accesses and adjacent threads often access adjacent memory (coalescing). Example: `A[row * K + col]` where both row and col are computed from thread/block indices.
Composite	A 1D flat index is unpacked into 2D (or higher) coordinates using integer division and modulo: `row = idx / N, col = idx % N`. Used when threads are linearized (e.g. `tid = ty·blockDim.x + tx`) but smem is indexed in 2D. Division and modulo cost extra instructions but do not add memory round-trips. This is the Flatten→Stride→Unflatten pattern used in cooperative smem loads: flatten thread coordinates into a single `tid`, stride it across the buffer, then unflatten back to 2D smem indices. Concrete example: for a tile `A_smem[BK][BM]` and thread index `tid`, the mapping is `row = tid / BM` (K dimension, 0..BK-1) and `col = tid % BM` (M dimension, 0..BM-1), giving `A_smem[row][col] = A[global_row * K + global_col]`.
Lookup	The physical address cannot be computed directly — we must first read an index array to obtain it: `global_array[ index_array[arithmetic_expr] ]`. Lookup leaves add a full extra memory round-trip (~200 cycles) before the

real data access and prevent hardware prefetching. Record them explicitly — each one surfaces as a design concern in Step 2 and as an indirection row in the index expression table. Example: `K[ kv_indices[row_ptr[q] + tile] · T_J + j ]` in sparse attention.

`— Value leaves —`	*Describe what happens to the value at the computed address.*
`Type conversion`	The data is stored in one numeric type but computed in another. A conversion instruction executes on every element access. Common in neural network kernels where weights are stored in FP8 or FP16 but computations occur in float32. When we identify a type-conversion leaf, note the storage type, the computation type, and the conversion — it will appear explicitly in the COMPUTE phase of the FSM. Example: `__half2float(A_fp16[row * K + col])`.
`Conditional`	A runtime condition determines whether the access happens at all: `condition ? value : default`. Two distinct placements follow — the placement is determined by the triggering iterator, not by the condition itself. Grid-level conditional → early exit (Bounds Check phase). If the triggering iterator is a grid-level dimension (blockIdx), place a cheap early-exit guard at the start of the kernel before any shared memory is touched. Tile-level conditional → predicated load (inside the tile loop). If the triggering iterator is a tile loop variable at Block or Warp granularity, the response is a predicated load or store inside the loop — never an early exit. An early exit here would leave other threads waiting at a subsequent `__syncthreads()` that is never reached, causing a deadlock.

> Compound index expressions. An index can contain multiple leaf types in the same expression — a lookup leaf wrapping composite and arithmetic leaves is common in paged systems. When this happens, classify the *entire access* by the highest-cost execution leaf present, using this priority: Lookup > Conditional > Type conversion > Composite > Arithmetic. A single lookup leaf makes the whole access a lookup-access regardless of the arithmetic around it. Record the lookup leaf explicitly because each one adds a full extra memory round-trip before the real data is reached.

Work through every tensor systematically and don't assume any dimension is arithmetic without checking. A dimension that appears simple at first might actually require a lookup, and missing that will mean missing a significant design concern before we've written any code. By the end of Step

1b, we have a complete picture of what your kernel needs to compute — the dimension fates, the chosen algorithm, the output contract, and the access structure of every tensor involved.

```
Feeds into final step → Index expression table (leaf type per access selects the formula column
— arithmetic AREA uses the cascade pattern, REDUCE into smem uses Flatten→Stride→Unflatten,
lookup adds an indirection row, conditional adds a predicate column)
```

STEP 2

The Binding Table

"Where in the GPU's execution hierarchy does each dimension live, and who is responsible for it?"

This is the most important design step in the framework. Everything up to this point was about understanding the problem. Step 2 is about designing the solution — specifically, assigning each dimension of your operation to a level of the GPU's execution hierarchy.

To fill in the Binding Table, we first need to understand what that hierarchy actually is. Think of it like a company with a very specific org chart, where each level has its own workspace that only people at that level can access.

Grid	Global mem	The whole GPU working on your problem. All blocks can read and write global memory, but it is large and slow — like a warehouse across town. Global memory is where inputs come from and where final outputs go. Blocks in the grid never communicate directly with each other during a kernel; they only coordinate through global memory before or after execution.
Block	Shared mem	A group of threads that share a fast private whiteboard — shared memory. It's much faster than global memory but small (typically 48–164 KB) and visible only within one block. When threads in the same block need to share intermediate results, shared memory is how they do it. Reading from shared memory after another thread wrote to it requires a `__syncthreads()` barrier.
Warp	Warp prims	Groups of exactly 32 threads that execute in hardware lockstep — when one thread executes an instruction, all 32 do simultaneously. This lockstep enables a family of collective operations — shuffles, votes, matches, and matrix ops — that work register-to-register with no shared memory and no

		barrier. Reductions are just one member of this family; the warp is a general collective execution unit.
Thread	Registers	Each individual worker with a private notepad — registers. Registers are the fastest memory on the chip and are completely private. State variables, loop accumulators, and intermediate values that belong to one thread's computation live here. The state variable you identified in Step 1a lives in registers during the REDUCE phase.

> What tiling means, and when a dimension supports it. Tiling means processing a large dimension in fixed-size chunks rather than all at once, so that each chunk fits in a faster memory level. A dimension supports tiling when processing one chunk either produces a complete independent result, or produces a partial result that combines correctly with partial results from other chunks. For a REDUCE dimension like K in GEMM, each tile of K contributes a partial dot product to the accumulator — the partial results sum correctly, so K is tileable. For an AREA dimension like M, each tile of M produces independent output rows that don't interact at all — trivially tileable. A dimension is *not* tileable when its chunks cannot be processed independently and cannot produce combinable partial results — for example, a scan dimension where each output depends on all prior outputs.

GRANULARITY — ONE QUESTION THAT APPLIES AT EVERY LEVEL

Before filling in the four column questions, fill in this table. Add one row for each dimension of your specific operation — carried forward from Step 1a — and record whether each level processes it as 1 (one element) or as tiled (a chunk whose size we must choose or is hardware-fixed). Dimensions of the same fate can tile very differently from each other, so each must be reasoned about individually. The Fate column is filled in from Step 1a; the rest we fill in now.

DIMENSION	FATE	GRID	BLOCK	WARP	THREAD
M your dim 1	AREA	all of M fixed	tiled budget check forces size	tiled or 1 hw-fixed if tensor cores	1 or can be tiled nothing forces this
N your dim 2	AREA	all of N fixed	tiled budget check forces size	tiled or 1 hw-fixed if tensor cores	1 or can be tiled nothing forces this
K your dim 3	REDUCE	all of K fixed	tiled budget check forces size	tiled or 1 hw-fixed if tensor cores	1 almost always 1

Replace M, N, K with your operation's actual dimensions — one row per dimension, in any order. For example, a convolution kernel might use `batch`, `channels`, `height`, `width`, and `kernel_size`; an attention kernel might use `query`, `key/value`, and `head_dim`. The table has one row per dimension from Step 1a, no more, no less. Two dimensions with the same AREA fate may still tile differently at the thread level: one may be too small to tile meaningfully, the other large enough to justify a register tile. The Fate column is what connects this table back to Step 1a; the per-row reasoning is what prevents blanket assumptions from hiding those differences. The thread column's free cells are the only ones with no external forcing mechanism — write a deliberate value there before moving on. Step 4b's Lifetime Table will verify it.

With granularities recorded, the Binding Table has four columns — one per level. The rows are your dimensions. For each (dimension, column) cell, you answer a specific question about what happens at that level's granularity. Work left to right across the columns for each dimension, because each column's answer often depends on the one to its left.

`Grid → Global`	Does a block permanently own a fixed slice of this dimension for the entire kernel's execution? *For AREA dimensions: usually yes. Assign blockIdx.x or blockIdx.y to this dimension. One block, one slice, owned forever. For REDUCE dimensions: usually no — the whole REDUCE dimension is processed within a single block.*
`Block → Shared`	Would multiple threads in the same block independently load the *same* data if we didn't stage it? *If yes: load that data once into shared memory. All threads read from the whiteboard instead of each making their own slow global memory trip. If no — if every thread reads unique data — staging is wasteful. The key word is "same."*
	What collective operation does the warp need to perform, and which primitive family implements it? *This is not only about reductions. Ask: are threads exchanging or permuting data (shuffles)? Making a collective yes/no decision (votes)? Finding lanes that share a value (match)? Multiplying matrix tiles (tensor cores)? Each has its own primitive family — all faster than shared memory and requiring no barrier.* Inter-warp collective trigger. After selecting the primitive family, compute `NUM_WARPS = (blockDim.x + 31) / 32` (ceiling division — always use ceiling, never floor) for every REDUCE dimension that uses a warp primitive. If `NUM_WARPS > 1`, the warp primitive alone is insufficient — each warp produces one partial result, and those partials must be combined across warps.

`Warp → Primitives`	This requires a shared memory merge buffer: add `warp_partial[NUM_WARPS]` (or `warp_partial[NUM_WARPS][tile_size]` for tiled reductions) to the Block→Shared column, and note that two additional `__syncthreads()` barriers will appear in the FSM — one after each warp's lane 0 writes its partial, one after thread 0 writes the final result. Include this buffer in the smem budget check below. Compile-time size requirement. Shared memory array dimensions must be compile-time constants in CUDA. `NUM_WARPS` must therefore be declared as a `constexpr` or derived from a template parameter or `#define` — it cannot be a runtime `int` computed inside the kernel. The canonical pattern is `constexpr int NUM_WARPS = (D_HEAD + 31) / 32;` placed before the kernel, where `D_HEAD` is itself a compile-time constant. A `static_assert(D_HEAD % 32 == 0, "...");` alongside it documents the assumption that blocks are always full warps. Active mask rule. Every warp primitive call requires an active mask as its first argument. Never hardcode `0xffffffff` — this assumes all 32 lanes are active, which fails for blocks where `blockDim.x` is not a multiple of 32. Instead, compute the mask explicitly: `unsigned mask = __ballot_sync(0xffffffff, true);` This returns the bitmask of all active threads participating in the ballot; when subsequently passed to `__shfl_down_sync`, only bits within the caller's warp are examined by hardware, so threads from other warps have no effect. Some developers prefer `unsigned mask = __activemask();` for clarity — both are correct here. The important invariant is that the mask passed to the shuffle matches the lanes that will participate in the reduction.
`Thread → Registers`	What does a single thread compute entirely on its own? *State variables identified in Step 1a live here. One thread's running sum, one thread's current maximum, one thread's heap. Private computation that no other thread needs to see during the COMPUTE phase.* Starting heuristic for thread tile size. A thread tile of 1 (one output element per thread) is always correct and a safe place to start. The only reason to increase it is if Step 4b will show that a register-held value is reused across the tile dimension — meaning we read it from shared memory once but use it multiple times before discarding it. If we're unsure, write 1 here and wait for Step 4b to tell us whether a larger tile is justified. The register budget check below is your ceiling: if the tile-1 design already uses 80+ registers, there is little room to grow.

> Warp primitive quick-match. If we're unsure which primitive family applies, match your computation to one of these patterns:
>
> "I need to sum / max / min values across all 32 lanes for the same output element." → Data Exchange (Shuffles), specifically `__shfl_down_sync` with offsets 16, 8, 4, 2, 1.
>
> "I need all lanes / any lane / how many lanes satisfy a condition." → Voting (`__all_sync`, `__any_sync`, `__ballot_sync`).
>
> "Multiple lanes might write to the same output bucket — I want to elect one representative per group to avoid serialized atomics." → Match (`__match_any_sync`).
>
> "My REDUCE dimension is a matrix multiply over tiles of shape 16×16×16 (or 16×8×16) and I want hardware tensor cores." → Warp Matrix Ops (`wmma::mma_sync`). Note that tensor cores fix the tile shape — this constrains your Block granularity in the Binding Table.
>
> "My REDUCE dimension is computed entirely within each thread's private registers and no lanes need to share partial results." → No warp primitive. GEMM with a register tile accumulator is the canonical example: each thread accumulates its own private C tile and never needs to exchange values with other lanes.

After filling in all four columns, do a shared memory budget check. Add up the sizes of everything we've marked as staged in shared memory. For each staged tensor or buffer, multiply the number of elements by the bytes per element. Sum all of it, including any `warp_partial` buffers generated by the inter-warp collective trigger above. If the total exceeds your hardware's shared memory limit per block (check your GPU's spec sheet — it varies), we have a problem that needs to be resolved now by adjusting tile sizes or reconsidering staging decisions. Finding this on paper is much better than finding it when your kernel silently fails to launch.

Immediately after, do a register budget check. Sum the sizes of all Thread→Registers state variables in float-equivalents. This is a lower bound — the compiler will add temporaries on top. If the sum exceeds 64 floats per thread, occupancy may fall below 4 warps per SM; if it exceeds 128, register spill into local memory (global memory with extra addressing) is likely. Both effects can cost more than the register tiling gains. Resolve by reducing thread-tile shape before proceeding to Step 4a.

Load coverage rule — verify before Step 3. For each shared memory buffer of size *S* elements, with *T* active threads in the block: each thread must load `ceil(S / T)` elements in the LOAD phase. One element per thread is only correct when S = T. The general pattern is a stride loop:

```
for (int p = threadIdx_linear; p < S; p += blockDim_linear) {
    smem[local_index(p)] = global[global_index(p)];
}
```

where `threadIdx_linear = ty·blockDim.x + tx` and the index formulas come from the Composite leaf pattern (Flatten→Stride→Unflatten). If your LOAD phase only loads one element per thread and S > T, it is a correctness bug: part of the smem buffer will contain uninitialized data. Verify the coverage equation for every staged buffer now, before placing barriers in Step 3.

Zero-length reduction guard. After computing `num_tiles` for any REDUCE dimension, add a guard: if `num_tiles == 0`, write the identity value and return immediately. Without this guard, a kernel operating on an empty sparse row or zero-length sequence will skip the tile loop but still execute NORMALIZE and STORE with uninitialized state variables. The identity for sum is 0, for max is −∞, for online softmax is m=−∞, l=0, o=0. Place this guard immediately after the tile count is computed, before the tile loop opens.

`Feeds into final step → Symbol legend (all dimension roles: grid index vars, block tile sizes, thread index vars, thread tile sizes, reg loop vars, memory sizes — one row per dimension) · Declaration table (launch config from Grid→Global; smem buffers from Block→Shared; register vars from Thread→Registers)`

The Warp level is the most commonly missed. Beginners often jump from Block to Thread, skipping Warp entirely. This leads to kernels that use shared memory and barriers for communication that could have happened through warp primitives for free — no memory consumed, no barrier latency. Always ask the Warp question explicitly, and ask it about the *specific nature* of the cooperation needed, not just "is there a reduction?"

The warp is not just a reduction engine. It is a collective execution unit with four distinct families of primitives. The right answer for the Warp column depends on which kind of cooperation your kernel actually needs. The first family is expanded — click any header to expand or collapse.

Data Exchange (Shuffles)	

`__shfl_sync`
`__shfl_up_sync`
`__shfl_down_sync`
`__shfl_xor_sync`
	Any thread needs a value from another lane's register — permutations, reductions, prefix operations, bitonic sort.

The shuffle family is the most general warp primitive. Each instruction lets a lane read the register value of any other lane directly, with no memory and no barrier. The four variants cover different communication patterns.

__shfl_sync(mask, val, src_lane) is a broadcast or gather: every lane specifies which lane it wants to receive from. Different lanes can pull from different sources simultaneously, making arbitrary within-warp permutations possible in a single instruction.

__shfl_down_sync(mask, val, delta) shifts data downward: lane N receives lane N+delta's value. This is the primitive most commonly seen in reductions — you shift by 16, then 8, then 4, then 2, then 1, and lane 0 accumulates the total.

__shfl_up_sync(mask, val, delta) shifts data upward: lane N receives lane N−delta's value. This is the natural primitive for prefix (scan) operations — each lane combines with its left neighbor.

__shfl_xor_sync(mask, val, lane_mask) exchanges between lanes whose IDs differ by a XOR pattern. Butterfly communication networks — which underlie bitonic sort, Walsh-Hadamard transforms, and many parallel algorithms — are wired exactly this way. Every compare-and-swap round of a bitonic sort within a warp is one __shfl_xor_sync call.

Use the shuffle family when threads need data from other lanes. Use __shfl_down for reduction, __shfl_up for prefix scans, __shfl_xor for sort networks and butterfly patterns, and __shfl_sync for arbitrary permutations.

`Voting`
`__ballot_sync`
`__all_sync` `__any_sync`
	The warp needs to make a collective decision about its own state — early termination, counting active lanes, finding which lanes satisfy a condition.

`Match`
`__match_any_sync`
`__match_all_sync`
	Multiple lanes need to update the same output position — histograms, sparse accumulations, gather/scatter where threads collide on the same destination.

`Warp Matrix Ops`
`wmma::load_matrix_sync`
`wmma::mma_sync`
`wmma::store_matrix_sync`
	The REDUCE dimension is a matrix multiply-accumulate and the tile sizes match Tensor Core dimensions (16×16×16 or similar).

STEP 4A

Lifetime Table — Block Level

"Does the reuse count for each Block→Shared buffer actually justify staging it? Validate before building the FSM."

Step 4a is a validation step that runs *before* the FSM is constructed. The Binding Table just made staging decisions — it said certain data should live in shared memory because multiple threads benefit from sharing it. Step 4a gives you a mechanical way to verify each of those decisions before you build barrier placements around them. A wrong staging decision discovered here costs one revision to Step 2. The same mistake discovered after Step 3 costs a revision to Step 2 *and* a complete redo of all barrier positions in the FSM.

The underlying principle: staging data into faster memory is only worthwhile if that data is reused enough times to justify the staging cost. If a value is written to shared memory and read exactly once, you paid the staging cost for zero benefit. If it is read a hundred times, you replaced a hundred slow global memory accesses with one global read and a hundred fast shared memory reads. The reuse count determines which case we are in.

> How to compute the reuse count for any staged value: Identify which loop iterators are active — still running — while this value is held in shared memory. Then identify which of those active iterators are *absent* from the value's own index expression. The absent iterators are the ones the value is reused across. Their combined sizes give the reuse count. A reuse count of 1 means staging provides no benefit; a reuse count of the block tile size or larger strongly justifies staging.

Work through every buffer in the Block→Shared column. For each one, compute the reuse count using the rule above. Then apply the check in both directions:

If we staged something in shared memory but its reuse count is 1 or very small, remove it from Block→Shared and revise Step 2 before continuing. The staging is consuming shared memory capacity and memory bandwidth for no gain — and it is reducing the number of blocks that can run concurrently on the SM, which costs occupancy.

If we left something in global memory but its reuse count is large, add it to Block→Shared and revise Step 2. We have missed a staging opportunity that would substantially improve performance.

> What counts as "large enough" to justify staging? A reuse count of 1 never justifies staging — you pay the smem write cost for exactly zero reads saved. A reuse count equal to the block tile size or larger (e.g. reuse = BN = 64 for a GEMM A tile) strongly justifies staging. For reuse counts in between, apply this rule: if the smem budget is not yet tight and global memory bandwidth is likely your bottleneck (common for memory-bound kernels), stage it. If the smem budget is already near the limit and the kernel is compute-bound, leave it in global memory. When genuinely unsure, stage it — the default behavior of hardware L2 cache provides a reuse benefit anyway, and explicit smem staging makes that benefit reliable and measurable.

If any revision is made to Step 2 as a result of this check, redo the shared memory and register budget checks before proceeding. Only when the Block→Shared column is validated should we move on to Step 3 to place barriers around it.

WHAT THIS STEP PRODUCES — ONE ROW PER BLOCK→SHARED BUFFER

BUFFER (FROM STEP 2)	ACTIVE ITERATORS WHILE HELD	ABSENT ITERATORS → REUSE COUNT	DECISION
`GEMM · A_smem[BK][BM]`	K tile loop, inner k loop, thread n-tile loop (TN steps)	*n-tile loop absent → reuse = TN = 8*	Keep ✓
`GEMM · B_smem[BK][BN]`	K tile loop, inner k loop, thread m-tile loop (TM steps)	*m-tile loop absent → reuse = TM = 8*	Keep ✓
`Sparse attn · K_smem[T_J][D]`	Sparse j loop (nnz blocks), inner k loop, v-thread loop (D threads)	*v-thread loop absent → reuse = D = 64. All D threads share same K rows for dot product.*	Keep ✓
`Sparse attn · score_smem[T_J]`	Sparse j loop, inner j-in-tile loop	*v dim absent → reuse = D = 64. Written once by thread 0, broadcast to all D threads.*	Keep ✓ (strongest)
`Sparse attn · kv_indices`	Sparse j loop pointer walk	*Reuse = 1 — each index consumed once per sparse iteration. Lookup leaf: staging adds a round-trip without benefit.*	Do not stage — read directly from global
your buffer	*which loops are running while this is held?*	*which of those are absent from its index?*	*keep / remove / move to global*

```
Feeds into final step → Declaration table (confirmed smem buffer sizes — use validated values not Step 2 estimates)
```

STEP 3

The FSM — Kernel Phases

"What is the temporal sequence of the kernel — what phases execute in what order, and where do barriers go?"

The Binding Table told us where data lives. The FSM (Finite State Machine) tells us the temporal sequence in which data moves between those locations. A kernel is not one long computation — it's a series of distinct phases, each one crossing a specific memory boundary or performing a specific operation. Reading the Binding Table from left to right and top to bottom, we can identify every boundary crossing, and each crossing becomes a phase.

The phases below are the standard ones. Your kernel will use some subset of them — only the ones the Binding Table actually requires. If nothing is staged in shared memory, there is no LOAD phase. If all merging is intra-warp, there is no BLOCK MERGE. The FSM is discovered from the Binding Table, not invented from scratch.

Novel phases. If your Algorithm Classification named a phase that does not appear in the standard list — a NORMALIZE phase, a SORT_PASS, a PREFIX_SCAN_UP, a HISTOGRAM_MERGE — insert it at the position the memory boundary crossings dictate. Name it descriptively. The standard phases are the most common cases, not an exhaustive list. Every novel phase still must answer the same three questions: what memory boundary does it cross, does that crossing require a barrier, and what dimension does threadIdx index during it?

—	barrier	dim binding →	Column headers. For every phase that uses a warp primitive, write "lane_id → [dim]". For every other phase that touches memory, write "threadIdx → [dim]". This column must be filled for all phases, not just warp phases — the thread index can serve different dimensional roles in different phases, and recording the binding for each phase is the only way to catch role-switching bugs before they reach code.
Bounds Check	—	—	Triggered by any conditional leaf in Step 1b. Check whether the thread's assigned position falls within valid data bounds. If not, write any required default value (often zero) and return immediately. This is cheap and

			prevents all subsequent work from being wasted on out-of-range threads.
Load	barrier	tile elements	Triggered when the Block→Shared column has any "yes" entries. Threads cooperatively load data from global memory into the shared memory staging area. Each thread loads a portion; together they fill the whole buffer. Lane_id strides across the tile's elements to maximize memory coalescing. A barrier is required after this phase because subsequent threads will read what their neighbors just wrote.
Compute	—	← declare	The main work. Each thread processes its portion of the REDUCE dimension and updates its register-level state variable. This is where we must write your lane binding declaration. Which problem dimension does lane_id index during compute — the output elements, or a slice of the reduction dimension? The answer changes what a subsequent warp reduction means. Everything else is handled by a loop inside each lane. Write it down before writing any code.
Warp Reduce / Warp Op	—	REDUCE dim	Triggered when the Warp→Primitives column has any entries. The 32 lanes perform their collective operation — a shuffle-based reduction, a prefix scan, a bitonic sort pass, a vote, a match. For a shuffle reduction to be correct, lane_id must index different elements of the same REDUCE dimension — each lane holds a partial result for the same output element, not a complete result for a different output element. No barrier needed. Each warp produces one result, typically held by lane 0.
Block Merge	barrier ×n	warp index	Triggered when threads from different warps need to combine their results. Lane 0 of each warp writes its result to the shared memory merge buffer. A barrier fires. Then a binary tree reduction across warps takes place — each round, half the warps read their partner's result from shared memory and merge it. One barrier is needed per round of the tree.
			The final phase, triggered by the output contract from the Algorithm Classification step. One thread (or a small coordinated group) writes the final result from registers

Store	—	—	to its position in global memory. If the output contract requires sorting, a sort phase precedes this one. No barrier needed after Store — threads write to unique positions they own permanently.

THE BARRIER RULE — ONE SENTENCE

Place a `__syncthreads()` barrier at every transition where thread A has written data into shared memory that thread B in the same block needs to read. If only one thread reads what it wrote, no barrier. If threads in different warps share the data, a barrier is required. Warp-level shuffles are exempt entirely — the lockstep execution of the warp is its own synchronization.

THE LANE BINDING RULE — ONE DECLARATION PER PHASE

For every phase that uses a warp primitive, write one sentence in your FSM: *lane_id indexes dimension ___*. This declaration specifies which problem dimension varies across the 32 lanes of the warp during this phase. Every other dimension is handled by a loop inside each lane. Writing it down is the entire rule — the value of doing so comes from two cross-checks it enables.

Cross-check one: consistency with the Compute phase. Compare the lane binding in your Compute phase with the lane binding in your Warp Reduce phase. If they differ, the shuffle is mixing partial results from different output elements rather than combining partial results for the same output element. This is the canonical form of the shuffle corruption bug — lanes compute complete dot products for different tokens, then the shuffle sums those independent dot products together as if they were partial sums for one dot product. The mismatch on paper is the signal. If they must differ (because the kernel genuinely changes what lanes represent between phases), we need a warp-level shuffle for data exchange before the reduction, not a reduction shuffle directly.

Cross-check two: consistency with the reduction character. Look at the dimension your lane binding declares in the Warp Reduce phase. Find that dimension in Step 1a. Since all REDUCE dimensions in this framework are parallel character, a shuffle reduction is valid — provided the lane binding is correct. If the cross-check fails here, the issue is with the lane binding assignment, not the reduction character.

THE THREAD PHASE-ROLE RULE — ONE DECLARATION PER NON-WARP PHASE

The dim-binding column must also be filled for phases that do not use a warp primitive. Write: *threadIdx indexes dimension ___*. This captures the fact that the same thread index can serve different dimensional roles in different phases. In a kernel where threadIdx.x acts as a key-dimension index during a score-computation phase and an output-dimension index during an accumulation

phase, the two phases have different thread roles — and unless both are declared, the mismatch is invisible on paper.

Cross-check three: consistency across shared memory boundaries. After declaring the thread role for every phase, find every pair of adjacent phases (A writes smem, B reads smem) and check that the index interpretation of the shared memory buffer is the same in A as in B. If phase A writes `smem[threadIdx.x]` with threadIdx.x meaning dimension k, and phase B reads `smem[threadIdx.x]` with threadIdx.x meaning dimension v, the buffer is populated in k-order and consumed in v-order — a silent indexing corruption. The fix is either to reindex the smem read in B, or to recognize that the two phases require a buffer transpose between them. This cross-check catches the bug on paper; without declared thread roles it can only be caught by inspecting a failing numerical output.

> Cross-check 3 — a concrete example. *Most simple kernels (vector add, reduced sum, GEMM) have a single consistent thread role throughout and will pass this check automatically. This example matters most when a kernel's phases assign threadIdx to different problem dimensions — most commonly in attention kernels where the head dimension is used for both dot-product computation and output accumulation.* Suppose a kernel has two phases:
>
> Phase A (LOAD): `smem[threadIdx.x] = K_global[j * D + threadIdx.x]`. Here threadIdx.x is being used as the head-dimension index *k*. A 64-thread block fills smem[0..63] with K[j, 0..63].
>
> Phase B (COMPUTE): the code then reads `smem[threadIdx.x]` while computing a dot product. threadIdx.x still means k. Same role. Cross-check passes.
>
> Now suppose Phase C (ACCUMULATE) also reads `smem[threadIdx.x]`, but this time threadIdx.x is the output-dimension index *v*. The developer intends to read "my V column." But smem was filled in k-order, not v-order. If k and v both happen to have size D = 64, the read is physically correct (smem[5] when threadIdx.x=5 means the same byte whether k=5 or v=5). Cross-check passes only because D_k = D_v.
>
> If D_k ≠ D_v — for example, in Multi-head Latent Attention where D_k = 128 and D_v = 64 — the same code reads smem[0..63] as v-indices, but smem was filled for k-indices 0..127. Thread 63 reads smem[63] intending V[j, 63], but smem[63] holds K[j, 63], which is only half the K vector. Cross-check 3 would have flagged this: the thread role switches from k-domain to v-domain at the smem boundary, and D_k ≠ D_v makes the physical addresses incompatible.

For each phase, record two things: which state variables are alive — that is, which values are being held and not yet consumed — and the lane binding for any phase that uses a warp primitive.

A state variable is alive from its initialization through every phase that updates or reads it, until the phase that finally writes it out to its destination. Tracking this prevents a class of bugs where a state variable is overwritten before its value is used, or consumed in the wrong phase. The lane binding prevents the class of bugs where correctly structured state variables are combined in the wrong way.

> Feeds into final step → Phase skeleton (read FSM rows top to bottom — each phase row becomes one code block in sequence; barrier column gives sync placement; novel phases from Algorithm Classification appear at their specified positions)

⏩ OPTIONAL — PERFORMANCE EXTENSION Read after Step 3 is complete and your kernel is correct

OPT 3+

Pipelining & Double Buffering

"My kernel is correct — but the memory unit sits idle while threads compute. How do I overlap the two?"

The FSM we built in Step 3 follows a safe, sequential rhythm: *load a tile into shared memory → barrier → compute on that tile → repeat*. During the compute phase, threads are busy doing arithmetic — but look at what else is happening, or rather, what isn't: the memory unit is completely idle. The GPU's ability to fetch data from global memory sits unused while threads crunch numbers. Then when compute ends and the next load begins, the arithmetic units sit idle while memory is busy. The two pipelines take turns rather than running simultaneously.

Pipelining is the technique that closes this gap. Instead of waiting for tile t's computation to finish before starting tile t+1's load, you fire off the load of tile t+1 in the background while computing on tile t. By the time the computation on tile t finishes, the load of tile t+1 is already done — or at least well underway. Memory and arithmetic run simultaneously rather than alternating.

This sounds simple, but it creates an immediate problem: we can't load tile t+1 into the same shared memory buffer we're currently computing on. Overwriting the buffer mid-computation would corrupt the results. The solution is to allocate two shared memory buffers that alternate roles — a "ping" and a "pong." While we compute on ping, we load

into pong. While we compute on pong, we load into ping. This pattern is called double buffering, and it's the only structural change to shared memory that pipelining requires.

The second ingredient is asynchronous copy, available on sm_80 and later via the `cp.async` instruction. A normal load blocks all threads in the block until the data arrives — that's what makes `__syncthreads()` after a load necessary. An async copy fires off the memory request and immediately lets threads continue without waiting. Think of it as handing the memory unit a shopping list and saying "get these while I start cooking." Threads proceed to the compute phase on the current tile, and only pause to wait for the async load when they actually need the freshly loaded tile at the top of the next iteration.

> Is pipelining worth the added complexity? Only when the compute phase is long enough to cover the load latency. If your load takes 200 cycles and your compute takes 20, the async load finishes long before compute ends — you'll see no benefit. If compute takes 500 cycles and load takes 200, pipelining can almost completely hide the load cost. The Lifetime Table (Step 4) gives you a signal before we write a single line: a high reuse count means compute time dominates, which means pipelining will help. A reuse count of 1 or 2 usually means compute is too short to care.

Step 2 extension — the Async column. If we decide to pipeline, we return to our Binding Table and add one new column: Async. For every shared memory buffer, record whether its loads will be synchronous (the baseline, write "no") or asynchronous (write "cp.async"). Most kernels will only mark the main data tiles as async — scratch buffers for warp merging, for instance, are usually not candidates. This keeps the decision visible and traceable: anyone reading your table later can immediately see which buffers were deliberately pipelined and which weren't. For a tiled GEMM where both operand tiles are pipelined, the extension looks like this:

Buffer	Block→Shared	Async
Tile A (M×K)	yes (staged)	cp.async — overlap with compute
Tile B (K×N)	yes (staged)	cp.async — overlap with compute

Step 3 extension — the pipelined FSM. The pipelined FSM has the same phases as the basic FSM, but two things change: there's a new Init phase before the main loop that

primes the first buffer, and the barrier type inside the loop shifts from `__syncthreads()` to `cp.async.wait_group(0)`. The underlying barrier rule is the same — don't read until the write is done — but the mechanism changes because the write (the load) is now happening asynchronously in the background. The wait at the end of each iteration ensures that by the time the next iteration's compute phase begins, the async load that was fired in the current iteration has finished landing in shared memory.

Init	—	tile elements	Before entering the tile loop, fire `cp.async` to load tile 0 into buffer 0, then immediately call `cp.async.wait_group(0)` to wait for it. This primes the pipeline. Buffer 0 is ready and waiting before the first iteration begins.
Load next (async)	—	tile elements	At the top of iteration t, fire `cp.async` to load tile t+1 into buffer `(t+1) % 2`. Don't wait — just issue the request and continue. The memory unit handles the transfer in the background while threads move on to compute.
Compute current	—	← declare	Compute on buffer `t % 2`, which holds tile t. This data was loaded by the previous iteration's async step (or by Init for t=0) and is guaranteed ready. Arithmetic and memory loads for tile t+1 are running simultaneously. Declare lane_id binding here — it must match the Warp Reduce phase that follows.
Sync	*async wait*	—	Call `cp.async.wait_group(0)`. This waits for the async load fired at the top of this iteration to finish before the next iteration's compute phase reads it. If compute took longer than the load — the ideal case — this wait returns immediately.
Store	—	—	After all tiles are processed, write the accumulated result from registers to global memory, exactly as in the basic FSM. This phase is completely unchanged by pipelining.

Notice what didn't change: the grid shape, the register state variables, the Lifetime Table reuse counts, the store phase, and the overall computation logic are all identical to the basic FSM. Pipelining is a targeted surgical extension — we add two buffers, shift the barrier type, and add an init phase. Everything else stays exactly where it was.

A way to verify pipelining actually helped. Open Nsight Compute after writing the pipelined kernel and find the "memory pipe utilization" and "compute pipe utilization" metrics. In a well-pipelined kernel, both should be high at the same time. If one flatlines while the other peaks, the two phases aren't overlapping — the compute phase is too short, or the tile size is too small for the overlap to be meaningful. The profiler will show you whether the pipelining worked without any guessing.

> Beginner note — save this for later. Pipelining is a performance optimization, not a correctness requirement. Your kernel already gives the right answer with the basic FSM from Step 3. The right time to come back to this section is after we've profiled your correct kernel and seen idle memory cycles in the output. At that point, you'll know exactly which buffer to pipeline, the Async column addition is a one-line change to your Binding Table, and the FSM extension is mechanical. Introducing pipelining before your kernel is correct is a reliable way to create race conditions that are very hard to debug — because a correctly pipelined wrong kernel looks exactly like an incorrectly pipelined correct one.

STEP 4B

Lifetime Table — Thread Level

"How many times is each register-held value reused, and does that justify the thread tile chosen in Step 2?"

Step 4b validates the Thread→Registers column of the Binding Table using the same reuse-count method applied to blocks in Step 4a, now applied one level down. The block-level staging decisions were already validated and locked in before the FSM was built. Step 4b now validates the thread-level tiling decisions — specifically, whether the thread tile shape chosen in Step 2's "free" column is actually earning its register cost.

The reuse count at the thread level works identically to the block level: identify which of the thread's owned AREA dimensions are absent from a given register-held value's index. Multiply

their per-thread sizes. That product is the register-level reuse count — the number of times one shared memory read is reused across the thread's register tile before the value is discarded.

> Thread-level reuse sub-table. For each register-held value, the active iterators are the thread's own loop counters over its owned tile dimensions plus any remaining REDUCE loops. The absent iterators among these are the reuse dimensions at the thread level. A concrete example: suppose a thread owns `TM` outputs along M and `TN` along N. It holds a loaded A element indexed by M and K — N is absent. As the thread iterates over its `TN` owned N outputs, that A element is reused `TN` times from registers without touching shared memory again. Register-level reuse count for A is `TN`. Similarly, a B element indexed by K and N has M absent, giving a register-level reuse count of `TM`.

The practical check: if the Binding Table's Thread→Registers column recorded a thread tile of granularity greater than 1 for any AREA dimension, verify here that at least one value held in registers has a non-trivial reuse count across that tile dimension. If no value does, the larger thread tile is consuming extra registers without benefit — reduce the granularity back toward 1 and recheck the register budget from Step 2. If a value does have a large reuse count but the thread tile was left at 1, we have missed a register tiling opportunity that would reduce shared memory traffic substantially — increase the thread tile shape and recheck the register budget.

Unlike Step 4a, corrections here do not require redoing the FSM. Barrier placement is driven by shared memory boundary crossings, which are set by the block-level staging decisions already validated. Thread tile changes affect only the inner compute loop and the register declarations in the Thread→Registers column. After making corrections, update Step 2 and proceed to Step 5.

WHAT THIS STEP PRODUCES — ONE ROW PER REGISTER-HELD VALUE

REGISTER VALUE	THREAD'S OWNED AREA DIMS	ABSENT AREA DIMS → REUSE COUNT	THREAD TILE JUSTIFIED?
`GEMM · a_reg[TM]`	m (TM elements), n (TN elements)	*n absent → reuse = TN = 8. One smem read reused 8× across n-tile.*	TM = 8 justified ✓
`GEMM · b_reg[TN]`	m (TM elements), n (TN elements)	*m absent → reuse = TM = 8. One smem read reused 8× across m-tile.*	TN = 8 justified ✓
`GEMM · C_acc[TM][TN]`	m (TM elements), n	*k absent → reuse = K/BK tiles. Accumulator persists*	Register accumulator justified ✓

REGISTER VALUE	THREAD'S OWNED AREA DIMS	ABSENT AREA DIMS → REUSE COUNT	THREAD TILE JUSTIFIED?
	(TN elements)	*across all K tiles.*	
`Sparse attn · q_reg`	v (1 element — thread tile = 1)	*j absent from Q index → reuse = nnz_blocks per query. One global load reused across entire sparse j loop.*	Register justified ✓; tile = 1 correct (v thread tile = 1)
`Sparse attn · o_acc`	v (1 element — thread tile = 1)	*j absent → reuse = nnz_blocks. Accumulator persists across all sparse KV blocks.*	Register justified ✓; Step 5 downward pass: tile > 1 possible but changes score layout — noted, not acted on
`Sparse attn · m_acc, l_acc`	v (1 element)	*j absent → reuse = nnz_blocks. Online softmax state persists across all sparse blocks.*	Register justified ✓
your register value	*which AREA dims does this thread own?*	*which are absent from this value's index?*	*increase tile / reduce tile / leave at 1*

Feeds into final step → Symbol legend (thread tile sizes confirmed — update any "TBD" entries from Step 2) · Declaration table (register accumulator shape, e.g. `float C_acc[TM][TN]`)

STEP 5

The Binary Tree Table (Index Trees)

"How is each memory address computed, and what does the structure of that computation reveal?"

The final step is the most mechanical one, and it serves two distinct purposes: verification and opportunity detection. Every time your kernel reads from or writes to memory, it computes an address. That address is always an expression built from execution indices and loop variables. Step 5 asks you to write each of those expressions as a tree, where the leaves are labeled with their type from Step 1b.

The tree doesn't need to be elaborate. The root is the full address. The branches split it into components. The leaves are the individual terms — each one labeled as arithmetic, lookup, type-

conversion, or conditional. A simple access like `data[batch * N + i]` produces a tree with two arithmetic leaves: one for `batch × N` and one for `i × 1`. A paged access adds a lookup leaf at the top before the arithmetic leaves within the page.

Verification. Once we have the tree, computing the maximum possible address is straightforward: substitute the maximum value for each leaf (for a dimension of size N, the maximum index is N−1) and evaluate the expression. Check that this maximum address falls within the allocated size of the corresponding buffer. Do this for every access in your kernel. It's tedious but mechanical, and it catches buffer overflow bugs before they happen — bugs that are notoriously difficult to track down once the code is running on a GPU.

Opportunity detection — upward pass. Look at your REDUCE dimensions — the ones that are contracted — and find them in the index trees. For each REDUCE dimension that appears as an arithmetic leaf, ask: *at what level of the execution hierarchy is this dimension currently being processed?* Then ask the follow-up question: *what would happen if it were processed at the next level up?*

If a REDUCE dimension currently owned by individual threads appears as an arithmetic leaf with a simple stride, moving it to the warp level would mean all 32 threads in a warp access 32 consecutive memory positions simultaneously. That's perfectly coalesced global memory access — one of the most significant optimizations available on a GPU. If a REDUCE dimension currently processed sequentially within a thread has no data dependencies between its iterations, it could be parallelized across warp lanes and reduced with shuffle instructions.

Opportunity detection — downward pass. Look at your AREA dimensions and find them in the index trees. For each AREA dimension that appears as an arithmetic leaf at Block or Warp granularity, ask: *what would happen if a slice of it were moved down into the thread's register tile?* If threadIdx currently addresses a single element of this dimension, and the kernel iterates over a REDUCE dimension while holding that output element in a register, then the output element is being reused exactly once per REDUCE step — which is the minimum. Increasing the thread tile to T elements along this AREA dimension would mean one shared memory read is reused T times across the tile, replacing T smem reads with T register reads for each loaded value. If the Step 4b reuse table already recorded a large thread-level reuse count for this dimension, a tile of T=1 is wasteful — the downward analysis makes the missed opportunity explicit. Record the recommended thread tile size and note that Step 2's thread-tile entry should be revisited before coding.

Deciding whether to act on an opportunity. The downward pass identifies opportunities; it does not mandate acting on them. Use this rule: if the Step 4b reuse count for the AREA dimension exceeds 4 *and* the register budget from Step 2 has more than 32 float-equivalents of headroom, increasing the thread tile is likely worthwhile — revise Step 2's thread-tile entry and recheck the register budget. If either condition is not met, note the opportunity in your derivation record and leave the tile at its current size. For the upward pass, the same logic applies: if moving a REDUCE dimension up one level requires introducing an inter-block reduction kernel or significant synchronization overhead, record the opportunity but assess whether the added complexity is warranted for your problem size.

These observations don't come from profiling. They come directly from reading the index tree structure. The framework makes these opportunities visible by construction, which is the deepest reason the Binary Tree Table exists.

BOUNDS VERIFICATION — ONE ROW PER GLOBAL MEMORY ACCESS

TENSOR ACCESS	MAX ADDRESS (SUBSTITUTE MAX INDEX PER DIM)	BUFFER SIZE	SAFE?
`GEMM · A[m, k]`	(M−1)×K + (K−1) = M×K − 1	*M × K floats*	✓
`GEMM · B[k, n]`	(K−1)×N + (N−1) = K×N − 1	*K × N floats*	✓
`GEMM · C[m, n]`	(M−1)×N + (N−1) = M×N − 1	*M × N floats*	✓
`Sparse attn · kv_indices[ptr]`	ptr ∈ [row_ptr[q], row_ptr[q+1]) → max = nnz_blocks − 1	*nnz_blocks integers*	✓ — must verify ptr never exceeds nnz_blocks
`Sparse attn · K[kv_idx·T_J+j, k]`	kv_idx via lookup leaf; max physical row = S_kv−1. Max addr = (S_kv−1)×D + (D−1) = S_kv×D − 1	*S_kv × D floats*	✓ — lookup leaf: verify kv_indices values are in [0, num_kv_blocks)
`Sparse attn · Q[q, k] / O[q, v]`	(S_q−1)×D + (D−1) = S_q×D − 1	*S_q × D floats*	✓
your access	*substitute max value for each dimension*	*allocated buffer size*	*✓ or ✗ (fix if ✗)*

OPPORTUNITY DETECTION — RECORD EVERY CANDIDATE, THEN DECIDE

DIMENSION	FATE	CURRENT LEVEL	DIRECTION	OPPORTUNITY	AC
GEMM · K	REDUCE	Thread (inner loop)	Up → Warp	*Warp shuffle reduction: 32 threads each take one k, reduce to lane 0. Eliminates private TM×TN tile.*	N pr ac gi = sh de th
GEMM · M, N	AREA	Block (smem)	Down → Thread	*Register tile TM×TN gives a_reg reuse = TN, b_reg reuse = TM.*	Ye im at ✓
Sparse attn · k (head dim)	REDUCE	Thread (warp shuffle)	Up → already at Warp	*__shfl_down_sync already implements warp-level k reduction. NUM_WARPS=2 → warp_partial inter-warp stage also derived.*	Al im ✓
Sparse attn · j (sparse KV blocks)	REDUCE	Block (sparse tile loop)	Up → Grid	*Multiple blocks process disjoint subsets of kv_indices, combine online-softmax states via global memory (split-KV). Requires atomic online-softmax merge or a second reduction kernel.*	N im — op fo sp se ac si co
Sparse attn · v (output dim)	AREA	Block/Thread (tile=1)	Down → Thread tile > 1	*Each thread could accumulate multiple v columns, amortizing score_smem broadcast across a v-tile. Requires*	N at co ou IL re la

DIMENSION	FATE	CURRENT LEVEL	DIRECTION	OPPORTUNITY	AC
				per-thread score storage — changes smem layout.	
your dim	*AREA / REDUCE*	*current hierarchy level*	*up / down*	*what changes and what is gained*	*ye re*

Feeds into final step → Index expression table (max address check column — mark ✓ or ✗ for each tensor access; any ✗ must be resolved before the skeleton is produced)

CODE DERIV.

Code Derivation

"Given the completed tables, produce the kernel skeleton mechanically — in the order your IDE needs it."

Four sub-tables, completed in the order shown. Each sub-table has a source column pointing to the prior step that provides each value. Complete sub-table 1 first — it establishes the variable names used throughout sub-tables 2, 3, and 4. Everything here is symbolic: no numeric constants appear except in the max-address check column of sub-table 3, where they serve as a verification sanity check and nowhere else.

1 OF 4 — SYMBOL LEGEND · POPULATE FROM THE STEP 2 GRANULARITY TABLE · ONE ROW PER DIMENSION

DIMENSION	FATE	GRID INDEX VAR	BLOCK TILE VAR	THREAD INDEX VAR	THREAD TILE VAR	REG LOOP VAR
GEMM · m	AREA	*blockIdx.y*	*BM*	*threadIdx.y*	*TM*	*tm*
GEMM · n	AREA	*blockIdx.x*	*BN*	*threadIdx.x*	*TN*	*tn*
GEMM · k	REDUCE	—	*BK*	—	— *(thread-private)*	*k*

DIMENSION	FATE	GRID INDEX VAR	BLOCK TILE VAR	THREAD INDEX VAR	THREAD TILE VAR	REG LOOP VAR
Sparse attn · q	AREA	`blockIdx.x`	*1 (one query per block)*	—	—	—
Sparse attn · v	AREA	—	`D_HEAD`	`threadIdx.x`	*1*	—
Sparse attn · k	REDUCE	—	`D_HEAD`	`threadIdx.x` *(warp lane)*	—	`d_idx`
Sparse attn · j	REDUCE	—	`T_J`	—	—	`j` *(sparse iter)*
your dimension	*AREA / REDUCE*	*blockIdx.x / .y / —*	*block tile constant*	*threadIdx.x / .y / —*	*thread tile constant or 1*	*loop variable name*

2 OF 4 — DECLARATIONS · EVERYTHING DECLARED BEFORE THE KERNEL BODY AND BEFORE THE TILE LOOP

WHAT TO DECLARE	SOURCE STEP	GEMM	SPARSE ATTENTION	YOUR
Launch grid	Step 2 Grid→Global; symbol legend grid index vars and block tile vars	`dim3 grid(ceil(N/BN), ceil(M/BM));`	`dim3 grid(S);`	*one bl AREA read fr legend*
Launch block	Step 2 granularity Thread row; block tile ÷ thread tile gives thread count per dim	`dim3 block(BN/TN, BM/TM);` → *(8, 8)*	`dim3 block(D_HEAD);` → *64 threads*	*block t thread AREA*
Smem buffers	Step 2 Block→Shared, validated by Step 4a; use confirmed sizes	`__shared__ float A_smem[2][BK][BM], B_smem[2][BK][BN];`	`__shared__ float K_smem[2][T_J][D_HEAD], V_smem[2][T_J][D_HEAD], score_smem[T_J], warp_partial[NUM_WARPS][T_J];`	*one declara Step 2 Block buffer (valida 4a)*

WHAT TO DECLARE	SOURCE STEP	GEMM	SPARSE ATTENTION	YOUR I
Register state vars	Step 1a state variable type and init value; Step 4b confirmed tile shape	`float C_acc[TM][TN] = {};` *(init = additive identity 0)*	`const float q_reg = Q[q_idx*D_HEAD+d_idx];` `float m_acc=-FLT_MAX, l_acc=0.f, o_acc=0.f;`	*one declara Step 1 variabl value f Step 1*
Tile loop bounds	Step 2 REDUCE dim Block granularity; memory size ÷ tile size from symbol legend	`int num_tiles = ceil(K / BK);`	`int num_tiles = row_ptr[q_block+1] - row_ptr[q_block];` *(sparse: from CSR)*	*memor var ÷ b var per REDU*

3 OF 4 — INDEX EXPRESSIONS · ONE ROW PER GLOBAL MEMORY ACCESS · LEAF TYPE FROM STEP 1B SELECTS THE ACTIVE COLUMN

TENSOR & DIM	LEAF TYPE	ARITHMETIC AREA GRID×BLOCK + THREAD×TILE + REG	ARITHMETIC REDUCE FLATTEN → STRIDE → UNFLATTEN	LOC ARR
GEMM · A[m,k] global	Arith. REDUCE (coop. smem load)	—	`tid=ty*BLOCK_X+tx;` `aRow=tid/BM;` `aCol=tile*BK+tid%BK;` `A[aRow*K+aCol]` *Flatten: ty×BLOCK_X+tx · Stride: tile×BK · Unflatten: /BM, %BK*	—
GEMM · B[k,n] global	Arith. REDUCE (coop. smem load)	—	`bRow=tid/BN;` `bCol=tile*BK+tid%BK;` `B[bRow*N+bCol]`	—
GEMM · C[m,n] store	Arith. AREA	`m_out = blockIdx.y*BM + threadIdx.y*TM + tm;` `n_out = blockIdx.x*BN + threadIdx.x*TN + tn;` `C[m_out*N + n_out]`	—	—

TENSOR & DIM	LEAF TYPE	ARITHMETIC AREA GRID×BLOCK + THREAD×TILE + REG	ARITHMETIC REDUCE FLATTEN → STRIDE → UNFLATTEN	LOO ARR
Sparse attn · Q[q,k] load	Arith. AREA	`Q[blockIdx.x * D_HEAD + threadIdx.x]` *q grid=blockIdx.x, block tile=1; v thread=threadIdx.x, tile=1*	—	—
Sparse attn · kv_indices[ptr]	Lookup	—	—	`kv` + *arith the*
Sparse attn · K[j,k] smem load	Arith. REDUCE (coop. smem load) + lookup indirection for j	—	`j_abs = kv_block_idx * T_J + j_in_tile;` `K_smem[buf][j_in_tile][d_idx] = K[j_abs*D_HEAD + d_idx]` *kv_block_idx from lookup row above; j_in_tile direct; d_idx=threadIdx.x*	—
Sparse attn · O[q,v] store	Arith. AREA	`O[blockIdx.x * D_HEAD + threadIdx.x]`	—	—
your tensor & dim	*from Step 1b leaf type table*	*grid_var×block_tile + thread_var×thread_tile + reg_var (from legend)*	*Flatten: ty×blockDim.x+tx · Stride: tile×tile_size+flat_offset · Unflatten: /smem_cols, %smem_cols*	*ind*

4 OF 4 — PHASE-ORDERED CODE SKELETON · READ FSM (STEP 3) TOP TO BOTTOM · EACH ROW IS ONE CODE BLOCK IN SEQUENCE

#	CODE BLOCK	CONTENT FROM	GEMM
1	Declarations block before kernel body	Sub-table 2 in full	`dim3 grid(ceil(N/BN), ceil(M/BM));` `dim3 block(BN/TN, BM/TM);` `__shared__ float A_smem[2][BK][BM];` `__shared__ float B_smem[2][BK][BN];` `float C_acc[TM][TN] = {};` `float a_reg[TM], b_reg[TN];`

#	CODE BLOCK	CONTENT FROM	GEMM
2	BOUNDS_CHECK early return	Step 1b grid-level conditional leaves; legend grid index vars and memory size vars	`if (block_m*BM >= M || block_n*BN >=` `  return;`

3	Tile loop open
	over REDUCE dim
		Sub-table 2 tile loop bounds; Step 2 REDUCE Block granularity; novel loop structure if lookup leaf (sparse iter walks kv_indices)	`int num_tiles = (K + BK - 1) / BK;`
			`for (int tile = 0; tile < num_tiles;`
			`  int buf = tile & 1;`

4	LOAD
	+
	`__syncthreads()`
		Sub-table 3 REDUCE rows (Flatten→Stride→Unflatten or direct); Step 1b conditional leaf predicates; Step 2 Block→Shared buffer names	`// Load coverage: S=BK×BM elems, T=BL`
			`// Each thread loads ceil(S/T) elemer`
			`const int BT = (BM/TM)*(BN/TN);`
			`for (int p = tid; p < BK*BM; p += BT)`
			`  int row_in_tile = p % BM;  // M`
			`  int col_in_tile = p / BM;  // K`
			`  int global_row = block_m*BM + row`
			`  int global_col = tile*BK  + col_`
			`  A_smem[buf][col_in_tile][row_in_t`
			`  (global_row<M && global_col<K`
			`  ? A[global_row*K + global_col`
			`}`
			`for (int p = tid; p < BK*BN; p += BT)`
			`  int row_in_tile = p / BN;  // K`
			`  int col_in_tile = p % BN;  // N`
			`  int global_row = tile*BK  + row_`
			`  int global_col = block_n*BN + col`
			`  B_smem[buf][row_in_tile][col_in_t`
			`  (global_row<K && global_col<N`
			`  ? B[global_row*N + global_col`
			`}`
			`__syncthreads();`

5	COMPUTE
	register inner loops
		Legend thread tile vars (inner loop bounds); sub-table 2 register state var names; Step 2 Thread→Registers accumulation body	`for (int k = 0; k < BK; ++k) {`
			`  for (int tm=0; tm<TM; ++tm)`
			`  a_reg[tm] = A_smem[buf][k][ty`
			`  for (int tn=0; tn<TN; ++tn)`
			`  b_reg[tn] = B_smem[buf][k][tx`
			`  for (int tm=0; tm<TM; ++tm)`
			`  for (int tn=0; tn<TN; ++tn)`
			`  C_acc[tm][tn] += a_reg[tm`
			`}`

#	CODE BLOCK	CONTENT FROM	GEMM
6	WARP REDUCE if Warp→Primitives non-empty	Step 2 Warp→Primitives primitive family; Step 3 lane binding (cross-checks 1+2)	*Not present — REDUCE k is thread-privat*
7	BLOCK MERGE if inter-warp trigger fired	Step 2 inter-warp trigger (NUM_WARPS > 1); warp_partial buffer from sub-table 2; Step 3 sync locations	*Not present — no warp primitive, so no int*
8	Novel contract phases from FSM phases column	Algorithm Classification FSM phases from contract; Step 1a state variable for the accumulation body	*Not present — GEMM output contract: ST*
9	Tile loop close + `__syncthreads()`	Step 3 barrier rule (smem consumed; next LOAD will overwrite)	`if (tile+1 < num_tiles) __syncthr` `}  // end tile loop`
10	NORMALIZE if in output contract	Algorithm Classification output contract; Step 1a state variable (what to transform)	*Not present*

11	STORE	Sub-table 3 AREA rows (arithmetic AREA formula); sub-table 3 predicate column if conditional leaf; Step 2 AREA ownership (no atomic)	`for (int tm=0; tm<TM; ++tm) {`
			`  for (int tn=0; tn<TN; ++tn) {`
			`  int m_out = block_m*BM + ty*TM +`
			`  int n_out = block_n*BN + tx*TN +`
			`  if (m_out<M && n_out<N)`
			`  C[m_out*N + n_out] = C_acc[tm`
			`  }`
			`}`

#	CODE BLOCK	CONTENT FROM	GEMM
your phase	*read FSM phase name and sequence position from Step 3*	*content source from FSM phase row; dim binding column names the threadIdx role for that block*	*write your kernel's code block here*

> The check before we open your editor. Every line we are about to write should trace to one cell in one of these four sub-tables. Launch config traces to sub-table 2 row 1. Every `__shared__` declaration traces to sub-table 2 row 3. Every index expression traces to sub-table 3. Every code block traces to a sub-table 4 row in the exact order shown. Every `__syncthreads()` traces to a row transition between a LOAD or BLOCK_MERGE phase and what follows it. If a line of code cannot be traced, either it is unjustified and should be removed, or your tables are incomplete and need to be revisited before coding.

What we have when the framework is complete. We have an architectural specification of your kernel derived from the mathematical operation, not from trial and error. We know the exact grid dimensions and how they map to your problem's AREA dimensions. We know precisely what data lives in shared memory, what lives in registers, and what is read directly from global memory — and we have a reuse-count justification for each of those decisions. We know the full phase sequence of the kernel and the location of every barrier. We have verified that no memory access can exceed its buffer's bounds.

What the framework does not give you. The framework deliberately stops at the architectural boundary. It tells us that a REDUCE dimension needs a heap accumulator — it doesn't tell us how to implement that heap. It tells us that an intra-warp reduction should use shuffles — it doesn't give you the shuffle sequence for a specific data type and warp size. It identifies that a particular loop is a candidate for vectorized loads — it doesn't tell us the right vector width for your specific GPU generation. These implementation details require knowledge of the specific hardware and algorithm, and they are best answered by domain expertise or by targeted profiling after the kernel is structurally correct.

A useful final check before coding. Before opening your editor, read your planned implementation mentally and ask, for each structural decision: which table entry justifies this? The grid shape should trace to the Grid→Global column. Each shared memory buffer should trace to the Block→Shared column and be validated by the Lifetime Table. Each barrier should trace to a

specific memory boundary crossing in the FSM. Each state variable should trace to Step 1a's reduction body. If we find a structural decision that can't be traced to a table entry, either the decision is unjustified and should be removed, or your tables are incomplete and we need to go back and find where it belongs. Every structural decision has a home in the framework. If it doesn't have one, that's important information.

FINAL OUTPUT

Complete Kernels — Annotated

Every line traced to its source table and phase. Green = phase header. Blue = source table. Gray = not present.

gemm.cu | sparse_attention.cu

```
// ╔═══════════════════════════════════════════════════════════════╗
// ║  Tiled GEMM  ·  C[M,N] = A[M,K] x B[K,N]                      ║
// ╚═══════════════════════════════════════════════════════════════╝

// ── DECLARATIONS [Sub-table 4, row 1] ──────────────────────────
// [Sub-table 2, smem buffers | Step 2 Block→Shared, validated Step 4a]
__shared__ float A_smem[2][BK][BM];
__shared__ float B_smem[2][BK][BN];

__global__ void gemm(const float* A, const float* B, float* C,
                     int M, int N, int K) {

    // [Symbol legend, grid/thread index vars | Step 2 Grid→Global]
    const int block_m = blockIdx.y,  block_n = blockIdx.x;
    const int tx = threadIdx.x,      ty = threadIdx.y;
    const int tid = ty * (BN/TN) + tx;

    // [Sub-table 2, register state vars | Step 1a: acc over k, init=0]
    float C_acc[TM][TN] = {};
    float a_reg[TM], b_reg[TN];

    // [Sub-table 2, tile loop bounds | symbol legend: K / BK]
    const int num_tiles = (K + BK - 1) / BK;

// ── BOUNDS CHECK [Sub-table 4, row 2] ──────────────────────────
    // [Sub-table 3, conditional predicate | Step 1b: grid-level leaf → early exit]
```

```
    if (block_m * BM >= M || block_n * BN >= N) return;

// ── TILE LOOP [Sub-table 4, row 3] ──────────────────────────────────
    // [Sub-table 2, tile loop bounds | Step 2 REDUCE Block granularity: K / BK]
    for (int tile = 0; tile < num_tiles; ++tile) {
        const int buf = tile & 1;

// ── LOAD [Sub-table 4, row 4] ───────────────────────────────────────
        // [Load coverage rule: stride loop — each thread loads ceil(S/T) elements]
        // [A is M×K: row_in_tile=p%BM (M-dim), col_in_tile=p/BM (K-dim)]
        const int BLOCK_THREADS = (BM/TM) * (BN/TN);
        for (int p = tid; p < BK * BM; p += BLOCK_THREADS) {
            int row_in_tile = p % BM;                       // M dimension (0..BM-1)
            int col_in_tile = p / BM;                       // K dimension (0..BK-1)
            int global_row  = block_m * BM + row_in_tile;  // M index into A
            int global_col  = tile   * BK + col_in_tile;  // K index into A
            // [Conditional leaf: tile-boundary predicated load]
            A_smem[buf][col_in_tile][row_in_tile] =  // smem layout: [K][M] matches __shar
                (global_row < M && global_col < K) ? A[global_row * K + global_col] : 0.f;
        }
        // [B is K×N: row_in_tile=p/BN (K-dim), col_in_tile=p%BN (N-dim)]
        for (int p = tid; p < BK * BN; p += BLOCK_THREADS) {
            int row_in_tile = p / BN;                       // K dimension (0..BK-1)
            int col_in_tile = p % BN;                       // N dimension (0..BN-1)
            int global_row  = tile   * BK + row_in_tile;  // K index into B
            int global_col  = block_n * BN + col_in_tile; // N index into B
            B_smem[buf][row_in_tile][col_in_tile] =
                (global_row < K && global_col < N) ? B[global_row * N + global_col] : 0.f;
        }
        // [Step 3 FSM, barrier | smem written by all → read by all in COMPUTE]
        __syncthreads();

// ── COMPUTE [Sub-table 4, row 5] ────────────────────────────────────
        // [Symbol legend: TM, TN, tm, tn | Step 4b: a_reg reuse=TN=8, b_reg reuse=TM=8]
        for (int k = 0; k < BK; ++k) {
            for (int tm=0; tm<TM; ++tm)
                a_reg[tm] = A_smem[buf][k][ty*TM+tm];
            for (int tn=0; tn<TN; ++tn)
                b_reg[tn] = B_smem[buf][k][tx*TN+tn];
            for (int tm=0; tm<TM; ++tm)
                for (int tn=0; tn<TN; ++tn)
                    C_acc[tm][tn] += a_reg[tm] * b_reg[tn];
```

```
        }
        // [Step 3 FSM, barrier | smem consumed; next LOAD will overwrite]
        if (tile + 1 < num_tiles) __syncthreads();

    } // end tile loop

    // WARP REDUCE — not present [row 6: k is thread-private, no primitive needed]
    // BLOCK MERGE — not present [row 7: no primitive → inter-warp trigger silent]
    // NORMALIZE   — not present [row 10: output contract = STORE only]

// ── STORE [Sub-table 4, row 11] ──────────────────────────────────────
    for (int tm=0; tm<TM; ++tm) {
        for (int tn=0; tn<TN; ++tn) {
            // [Sub-table 3, arithmetic AREA | symbol legend: grid×BM + thread×TM + reg]
            int m_out = block_m*BM + ty*TM + tm;
            int n_out = block_n*BN + tx*TN + tn;
            // [Sub-table 3, conditional predicate | Step 1b: tile-boundary guard on store
            if (m_out < M && n_out < N)
                C[m_out*N + n_out] = C_acc[tm][tn];
        }
    }
}

// [Sub-table 2, launch config | Step 2 Grid→Global: n→blockIdx.x, m→blockIdx.y]
void launch_gemm(float* A, float* B, float* C, int M, int N, int K) {
    dim3 grid((N+BN-1)/BN, (M+BM-1)/BM);   // [symbol legend: n→x, m→y]
    dim3 block(BN/TN, BM/TM);                // [symbol legend: block tile / thread tile]
    gemm<<<grid, block>>>(A, B, C, M, N, K);
}
```

gemm.cu | sparse_attention.cu

```
// ╔═══════════════════════════════════════════════════════════════════════╗
// ║  Block-Sparse Flash Attention  (DeepSeek sparse pattern)            ║
// ║  O[q,v] = sparse_softmax_j( Q·K/sqrt(D) ) · V                       ║
// ╚═══════════════════════════════════════════════════════════════════════╝

// ── DECLARATIONS [Sub-table 4, row 1] ──────────────────────────────────
// [NUM_WARPS must be a compile-time constant — shared memory arrays require constant size
// [Because warp_partial[NUM_WARPS][T_J] is __shared__, NUM_WARPS must be constexpr, not a
```

```
// [D_HEAD and T_J are template parameters or #defines; NUM_WARPS is derived at compile ti
static_assert(D_HEAD % 32 == 0, "D_HEAD must be a multiple of 32 for full warps");
constexpr int NUM_WARPS = (D_HEAD + 31) / 32;  // compile-time constant; ceiling division

// [Sub-table 2, smem buffers | Step 2 Block→Shared, validated Step 4a]
__shared__ float K_smem[2][T_J][D_HEAD];
__shared__ float V_smem[2][T_J][D_HEAD];
__shared__ float score_smem[T_J];              // reuse=D_HEAD, highest-reuse buffer
// [Step 2 inter-warp trigger: NUM_WARPS=(D_HEAD+31)/32 → warp_partial required]
__shared__ float warp_partial[NUM_WARPS][T_J]; // NUM_WARPS constexpr above — valid smem s

__global__ void sparse_attention(
    const float* Q, const float* K, const float* V, float* O,
    const int* kv_indices, const int* row_ptr, int S)
{
    // [Symbol legend, grid/thread vars | Step 2 Grid→Global: q→blockIdx.x, k/v→threadIdx.
    const int q_idx   = blockIdx.x;
    const int d_idx   = threadIdx.x;
    // [NUM_WARPS is a constexpr defined above — not redefined here; warp/lane derived fro
    const int warp_id = d_idx / 32;
    const int lane_id = d_idx % 32;

// ── BOUNDS CHECK [Sub-table 4, row 2] ──────────────────────────────
    // [Sub-table 3, conditional predicate | Step 1b: grid-level leaf → early exit]
    if (q_idx >= S || d_idx >= D_HEAD) return;

// ── INIT [Sub-table 4, row 1 continued] ────────────────────────────
    // [Sub-table 3, arithmetic AREA | Q: blockIdx.x×D_HEAD + threadIdx.x]
    const float q_reg = Q[q_idx * D_HEAD + d_idx];
    // [Sub-table 2, register state vars | Step 1a: (m,l,o) over j; Algorithm Classificati
    float m_acc = -FLT_MAX;
    float l_acc = 0.f;
    float o_acc = 0.f;

    // [Sub-table 2, tile loop bounds | CSR row_ptr gives sparse tile count]
    const int q_block   = q_idx;
    const int num_tiles = row_ptr[q_block+1] - row_ptr[q_block];

    // [Zero-length guard | if sparse row is empty, write identity and return]
    if (num_tiles == 0) { O[q_idx * D_HEAD + d_idx] = 0.f; return; }

// ── TILE LOOP [Sub-table 4, row 3] ─────────────────────────────────
```

```
    // [Sub-table 3, lookup leaf | kv_indices drives iteration — not sequential j]
    for (int tile = 0; tile < num_tiles; ++tile) {
        const int buf = tile & 1;
        // [Sub-table 3, lookup | kv_indices[row_ptr[q]+tile] → physical KV block]
        const int kv_block = kv_indices[row_ptr[q_block] + tile];

// ── LOAD [Sub-table 4, row 4] ──────────────────────────────────────────
        // [Sub-table 3, arithmetic REDUCE + lookup | j_abs from lookup, d_idx direct]
        for (int j = 0; j < T_J; ++j) {
            int j_abs = kv_block * T_J + j;
            // [Sub-table 3, conditional predicate | Step 1b: tile-boundary guard]
            K_smem[buf][j][d_idx] = (j_abs < S) ? K[j_abs*D_HEAD+d_idx] : 0.f;
            V_smem[buf][j][d_idx] = (j_abs < S) ? V[j_abs*D_HEAD+d_idx] : 0.f;
        }
        // [Step 3 FSM, barrier | K_smem, V_smem written; read by all threads in COMPUTE]
        __syncthreads();

// ── COMPUTE [Sub-table 4, row 5] ───────────────────────────────────────
        for (int j = 0; j < T_J; ++j) {
            // [Sub-table 4 row 5 | q_reg reuse=nnz_blocks across tile loop, Step 4b]
            float partial = q_reg * K_smem[buf][j][d_idx];

// ── WARP REDUCE [Sub-table 4, row 6] ───────────────────────────────────
            // [Active mask rule: compute from ballot, never hardcode 0xffffffff]
            // [Step 2 Warp→Primitives: shfl_down | Step 3 cross-check 1: lane_id=k in COM
            unsigned mask = __ballot_sync(0xffffffff, true);
            for (int off = 16; off > 0; off >>= 1)
                partial += __shfl_down_sync(mask, partial, off);

// ── BLOCK MERGE write [Sub-table 4, row 7] ─────────────────────────────
            // [Step 2 inter-warp trigger: NUM_WARPS=(D_HEAD+31)/32 → warp_partial buffer]
            if (lane_id == 0) warp_partial[warp_id][j] = partial;
        }
        // [Step 3 FSM, barrier | warp_partial written by lane 0; thread 0 reads all]
        __syncthreads();

// ── BLOCK MERGE reduce + score [Sub-table 4, row 7 continued] ──────────
        if (d_idx == 0) {
            for (int j = 0; j < T_J; ++j) {
                float total = 0.f;
                for (int w = 0; w < NUM_WARPS; ++w) total += warp_partial[w][j];
                int j_abs = kv_block * T_J + j;
```

```
                score_smem[j] = (j_abs < S) ? total * rsqrtf((float)D_HEAD) : -FLT_MAX;
            }
        }
        // [Step 3 FSM, barrier | score_smem written by thread 0; read by all in SOFTMAX_U
        __syncthreads();

// ── NOVEL PHASE: SOFTMAX_UPDATE [Sub-table 4, row 8] ──────────────────
        // [Algorithm Classification: online softmax from output contract]
        // [Step 1a: state vars m_acc, l_acc, o_acc; reduce body = online softmax merge]
        for (int j = 0; j < T_J; ++j) {
            float s_j   = score_smem[j];
            float m_new = fmaxf(m_acc, s_j);
            float e_old = __expf(m_acc - m_new);
            float e_new = __expf(s_j  - m_new);
            l_acc = l_acc * e_old + e_new;
            o_acc = o_acc * e_old + e_new * V_smem[buf][j][d_idx];
            m_acc = m_new;
        }

        // [Step 3 FSM, barrier | smem consumed; next LOAD will overwrite K_smem, V_smem]
        __syncthreads();

    } // end tile loop

// ── NORMALIZE [Sub-table 4, row 10] ────────────────────────────────────
    // [Algorithm Classification: output contract → weights sum to 1 → divide by l_acc]
    o_acc /= l_acc;

// ── STORE [Sub-table 4, row 11] ────────────────────────────────────────
    // [Sub-table 3, arithmetic AREA | symbol legend: q→blockIdx.x×D_HEAD + threadIdx.x]
    O[q_idx * D_HEAD + d_idx] = o_acc;   // no predicate: S blocks cover q exactly
}

// [Sub-table 2, launch config | Step 2 Grid→Global: AREA q → one block per query]
void launch_sparse_attention(float* Q, float* K, float* V, float* O,
                             int* kv_indices, int* row_ptr, int S) {
    dim3 grid(S);        // [symbol legend: q → blockIdx.x, block tile = 1 query per block
    dim3 block(D_HEAD);  // [symbol legend: k/v → threadIdx.x, D_HEAD threads = 2 warps]
    sparse_attention<<<grid, block>>>(Q, K, V, O, kv_indices, row_ptr, S);
}
```

Quick reference

Step 1a Dimension Fate	For each dimension: does it appear in the output? Mark AREA or REDUCE. For every REDUCE dimension, write the reduction body and state variable. Verify the reduction is associative and commutative (parallel character). If it is not — for example, selection (top-K, argmax) or scan (prefix sum) — this framework does not apply; stop and use a dedicated algorithm reference for that operation. → dimension fate table, reduction bodies, state variable types, reduction characters
Algorithm Classification	Which algorithm family implements this reduction? Compute relevant ratios and choose. Write down the output contract — ordering, normalization, stochastic steps — everything the output must satisfy beyond shape. Name every intermediate tensor the chosen family introduces. → algorithm family, output contract, any intermediate tensors introduced
Step 1b Access Patterns	For each dimension of each tensor (including intermediates from the algorithm): what leaf type is the index? Arithmetic, lookup, type-conversion, or conditional? For conditional leaves: is the triggering iterator a grid-level dimension (→ Bounds Check phase, early exit) or a tile iterator (→ predicated load inside the tile loop, never an early exit)? → leaf type table; conditional leaf placement (early exit vs predicated load)
Step 2 Binding Table	Record granularity per (dimension, level). Fill four columns: Grid→Global, Block→Shared, Warp→Primitives, Thread→Registers. For Warp→Primitives: after selecting the primitive family, compute NUM_WARPS = (blockDim.x + 31) / 32 (ceiling division) per REDUCE dimension. If NUM_WARPS > 1, add a warp_partial[NUM_WARPS] buffer to Block→Shared and note two extra FSM sync points. Then: (1) smem budget check — sum all Block→Shared buffers including warp_partial; (2) register budget check — sum all Thread→Registers state variables, flag if >64 floats. → granularity chain, grid shape, smem layout + size, warp_partial if needed, register size estimate, warp primitive family
Step 4a Lifetime (block)	Before building the FSM: for every Block→Shared buffer, compute reuse count (product of active iterator sizes absent from the buffer's

index). If reuse = 1 and no coalescing justification, remove from Block→Shared and revise Step 2. If large reuse was missed, add to Block→Shared. Redo budget checks. Only proceed to Step 3 when all staging decisions are validated.

→ block-level reuse counts; validated Block→Shared column; any Step 2 revisions resolved before FSM

Step 3
FSM

Read the validated Binding Table for memory boundary crossings to identify phases. Apply the barrier rule: __syncthreads() wherever one thread writes shared memory that another thread reads. For every phase, fill the dim-binding column: "lane_id → [dim]" for warp-primitive phases; "threadIdx → [dim]" for all other phases that touch memory. Apply three cross-checks: (1) lane_id binding matches between Compute and Warp Reduce? (2) REDUCE dimension is parallel in character? (3) threadIdx role is consistent across each shared memory boundary — the buffer is populated and consumed with the same index interpretation? Record which state variables are alive in each phase.

→ ordered phase sequence, barrier locations, dim-binding column (lane_id + thread role), three cross-checks, state variable lifetimes

---

Opt · Pipeline (after Step 3)

Only if the Step 4a reuse count is high and profiling shows idle memory cycles: add an Async column to the Binding Table, allocate two shared memory buffers (ping/pong), replace __syncthreads() with cp.async.wait_group(), and add an Init phase before the tile loop.

→ async flag in binding table, double-buffered smem, pipelined FSM — memory and compute overlapped

---

Step 4b
Lifetime (thread)

After the FSM: for every register-held value, identify which of the thread's owned AREA dimensions are absent from its index. That product is the register-level reuse count. Validate the thread tile shape from Step 2: if tile > 1 but no register value has reuse > 1, reduce tile; if tile = 1 but reuse is large, increase tile. Corrections here do not require redoing the FSM.

→ thread-level reuse counts, validated Thread→Registers tile shape, any Step 2 thread-tile corrections

---

Step 5
Index Trees

Write every memory address as a tree of typed leaves. Compute max addresses and verify they fit allocated buffers. Upward pass: for every REDUCE leaf, ask what happens if it moves up one hierarchy level (parallelization opportunity). Downward pass: for every AREA leaf at Block or Warp level, ask what happens if a slice moves down into the thread register tile (ILP / register tiling opportunity). Record both sets of opportunities by name before opening an editor.

→ address range verification, upward parallelism opportunities, downward register-tiling opportunities · feeds into: Index expression table max address column

---

Code Derivation

Four sub-tables completed in sequence, entirely from the prior steps. (1) Symbol legend: one row per dimension, mapping abstract roles (grid index var, block tile, thread index var, thread tile, reg loop var, memory size) to your kernel's actual variable names — populated from the Step 2 granularity table. (2) Declaration table: launch config, smem buffers, register state vars — read from symbol legend and Steps 2/1a. (3) Index expression table: one row per global memory access, one active column per leaf type — arithmetic AREA uses the cascade formula from the legend, REDUCE uses Flatten→Stride→Unflatten, lookup adds an indirection, conditional adds a predicate. (4) Phase-ordered code skeleton: read FSM rows top to bottom; each row becomes one code block in the sequence shown.

→ symbol legend · declaration table · index expression table with max-address checks · phase-ordered code skeleton

---

The framework answers *"what structure should this kernel have?"* — grid shape, memory staging, phase ordering, barrier placement, warp usage. It does not answer *"how fast will it be?"* Occupancy, register spill counts, instruction throughput, and vectorization width are profiling-time concerns. The framework's scope ends exactly where systematic derivation gives way to hardware-specific tuning.

SCOPE LIMITATIONS

I've used this framework on kernels where every REDUCE dimension is associative and commutative (parallel reduction character). The kernels I've worked through with it are: vector add, transpose, shared reduced sum, GEMM, dense attention, paged KV MLA attention, and sparse attention.

It does *not* cover: scan (prefix sum, exclusive scan, running count — Blelloch up-sweep/down-sweep changes the phase structure to two passes with fundamentally different inter-thread dependencies); selection (top-K, argmax, beam search — requires warp min-heaps, radix select, or bitonic sort, each introducing intermediate tensors the Binding Table is not designed to derive); or any operation whose reduction body is not freely associative and commutative.

If the operation is outside this set, the derivation steps will still run but the phase structure they produce probably won't be right. Catching that at Step 1a saves a lot of rework later.